

Методичка 12. Dataclasses и сущности

1. Зачем нужна эта тема

`dataclass` нужен, когда в программе нужно удобно хранить данные об одном объекте.

Например, программа может работать с телефонами:

- название модели;
- бренд;
- цена;
- количество памяти;
- год выпуска.

Можно хранить эти данные в отдельных переменных:

```
brand = "Samsung"
model = "A55"
price = 35000
memory = 128
```

Но если телефонов много, отдельных переменных станет слишком много. Ещё хуже — если для каждого телефона нужно хранить несколько характеристик.

Можно использовать список:

```
phone = ["Samsung", "A55", 35000, 128]
```

Но такой код плохо читается. Непонятно, что означает каждый элемент списка.

`dataclass` позволяет создать понятную сущность:

```
@dataclass
class Phone:
    brand: str
    model: str
    price: int
    memory: int
```

Теперь телефон можно хранить как один объект с понятными полями.

Эта тема помогает перейти от простых списков к более структурированному коду. Она готовит к ООП, классам и объектам.

2. Главная идея темы

`dataclass` — это удобный способ описать сущность с полями.

Сущность — это объект из предметной области программы.

Например:

- телефон;
- книга;
- ученик;
- товар;
- фильм;
- машина;
- заказ;
- игрок.

У сущности есть характеристики. В программировании их часто называют полями.

Сущность — объект, о котором программа хранит данные.

Поле — характеристика сущности.

Класс — описание того, какие поля будут у сущности.

Объект — конкретный экземпляр класса.

`dataclass` — специальный инструмент Python, который помогает быстро создавать классы для хранения данных.

3. Базовый синтаксис

Импорт `dataclass`

```
from dataclasses import dataclass
```

Перед использованием `dataclass` его нужно импортировать.

Описание сущности

```
from dataclasses import dataclass

@dataclass
class Phone:
    brand: str
    model: str
    price: int
```

Что происходит:

- `@dataclass` говорит Python, что этот класс предназначен для хранения данных;
- `class Phone:` создаёт новый тип данных;
- `brand`, `model`, `price` — поля;
- после двоеточия указывается тип поля.

Создание объекта

```
phone = Phone("Samsung", "A55", 35000)

print(phone)
```

Python автоматически создаёт удобный способ создать объект.

Обращение к полям

```
phone = Phone("Samsung", "A55", 35000)

print(phone.brand)
print(phone.model)
print(phone.price)
```

Через точку можно получить значение поля.

Изменение поля

```
phone = Phone("Samsung", "A55", 35000)

phone.price = 32000

print(phone)
```

Поле можно изменить, если это нужно программе.

Список объектов

```
phones = []

phones.append(Phone("Samsung", "A55", 35000))
phones.append(Phone("Apple", "iPhone 13", 60000))

for phone in phones:
    print(phone.brand, phone.model, phone.price)
```

Список может хранить не только числа и строки, но и объекты.

4. Разбор темы от простого к сложному

Шаг 1. Проблема отдельных переменных

```
brand1 = "Samsung"
model1 = "A55"
price1 = 35000

brand2 = "Apple"
model2 = "iPhone 13"
price2 = 60000
```

Такой код неудобен. Если товаров станет 100, переменных станет слишком много.

Шаг 2. Проблема списка без названий

```
phone = ["Samsung", "A55", 35000]

print(phone[0])
print(phone[1])
print(phone[2])
```

Код работает, но плохо читается. Нужно помнить, что индекс 0 — бренд, индекс 1 — модель, индекс 2 — цена.

Шаг 3. Описание dataclass

```
from dataclasses import dataclass

@dataclass
class Phone:
    brand: str
    model: str
    price: int
```

Теперь структура данных описана явно.

Шаг 4. Создание объекта

```
phone = Phone("Samsung", "A55", 35000)

print(phone)
```

Объект содержит все данные об одном телефоне.

Шаг 5. Чтение полей

```
phone = Phone("Samsung", "A55", 35000)

print("Бренд:", phone.brand)
print("Модель:", phone.model)
print("Цена:", phone.price)
```

Поле читается через точку.

Это понятнее, чем `phone[0]`, `phone[1]`, `phone[2]`.

Шаг 6. Изменение поля

```
phone = Phone("Samsung", "A55", 35000)

phone.price = 33000

print(phone.price)
```

Если цена изменилась, можно изменить поле объекта.

Шаг 7. Список объектов

```
phones = [  
    Phone("Samsung", "A55", 35000),  
    Phone("Apple", "iPhone 13", 60000),  
    Phone("Xiaomi", "Redmi Note", 20000)  
]  
  
for phone in phones:  
    print(phone.brand, phone.model, phone.price)
```

Так можно хранить много сущностей.

Шаг 8. Поиск объекта

```
phones = [  
    Phone("Samsung", "A55", 35000),  
    Phone("Apple", "iPhone 13", 60000),  
    Phone("Xiaomi", "Redmi Note", 20000)  
]  
  
target_brand = input("Введите бренд: ")  
  
for phone in phones:  
    if phone.brand == target_brand:  
        print(phone)
```

Можно искать объекты по полям.

Шаг 9. Фильтрация объектов

```
phones = [  
    Phone("Samsung", "A55", 35000),  
    Phone("Apple", "iPhone 13", 60000),  
    Phone("Xiaomi", "Redmi Note", 20000)  
]  
  
for phone in phones:  
    if phone.price <= 40000:  
        print(phone)
```

Так можно вывести только подходящие объекты.

Шаг 10. Функция для вывода объекта

```
def print_phone(phone):  
    print("Бренд:", phone.brand)  
    print("Модель:", phone.model)  
    print("Цена:", phone.price)  
  
phone = Phone("Samsung", "A55", 35000)  
  
print_phone(phone)
```

`dataclass` удобно использовать вместе с функциями.

5. Частые ошибки учеников

Ошибка 1. Забыли импортировать `dataclass`

Неправильно:

```
@dataclass  
class Phone:  
    brand: str  
    model: str  
    price: int
```

Почему это ошибка:

Python не знает, что такое `dataclass`, если его не импортировать.

Правильно:

```
from dataclasses import dataclass  
  
@dataclass  
class Phone:  
    brand: str  
    model: str  
    price: int
```

Ошибка 2. Забыли символ `@`

Неправильно:

```
from dataclasses import dataclass

dataclass
class Phone:
    brand: str
    model: str
    price: int
```

Почему это ошибка:

`@dataclass` — это декоратор. Символ `@` обязателен.

Правильно:

```
from dataclasses import dataclass

@dataclass
class Phone:
    brand: str
    model: str
    price: int
```

Ошибка 3. Путают класс и объект

Неправильно:

```
print(Phone.brand)
```

Почему это ошибка:

`Phone` — это описание. Конкретные данные находятся в объекте.

Правильно:

```
phone = Phone("Samsung", "A55", 35000)

print(phone.brand)
```

Ошибка 4. Передают не все значения при создании

Неправильно:

```
phone = Phone("Samsung", "A55")
```

Почему это ошибка:

В классе описаны три поля: `brand`, `model`, `price`. Значит, при создании нужно передать три значения.

Правильно:

```
phone = Phone("Samsung", "A55", 35000)
```

Ошибка 5. Путают порядок аргументов

Неправильно:

```
phone = Phone(35000, "Samsung", "A55")
```

Почему это ошибка:

Значения переданы не в том порядке. Цена попадёт в бренд, бренд — в модель.

Правильно:

```
phone = Phone("Samsung", "A55", 35000)
```

Ещё понятнее можно так:

```
phone = Phone(brand="Samsung", model="A55", price=35000)
```

Ошибка 6. Пытаются обращаться как к словарю или списку

Неправильно:

```
phone = Phone("Samsung", "A55", 35000)

print(phone["brand"])
```

Почему это ошибка:

Поля объекта читаются через точку, а не через квадратные скобки.

Правильно:

```
phone = Phone("Samsung", "A55", 35000)

print(phone.brand)
```

Ошибка 7. Делают несколько списков вместо списка объектов

Неправильно:

```
brands = ["Samsung", "Apple"]
models = ["A55", "iPhone 13"]
prices = [35000, 60000]
```

Почему это неудобно:

Данные одного телефона разбросаны по разным спискам. Легко перепутать индексы.

Правильно:

```
phones = [
    Phone("Samsung", "A55", 35000),
    Phone("Apple", "iPhone 13", 60000)
]
```

6. Мини-примеры для закрепления

Пример 1. Простая сущность

```
from dataclasses import dataclass

@dataclass
class Book:
    title: str
    author: str
    year: int

book = Book("Хоббит", "Толкин", 1937)

print(book)
```

Закрепляется создание `dataclass`.

Пример 2. Обращение к полям

```
book = Book("Хоббит", "Толкин", 1937)

print(book.title)
print(book.author)
print(book.year)
```

Закрепляется доступ через точку.

Пример 3. Изменение поля

```
book = Book("Хоббит", "Толкин", 1937)

book.year = 1938

print(book)
```

Закрепляется изменение поля объекта.

Пример 4. Список объектов

```
books = [
    Book("Хоббит", "Толкин", 1937),
    Book("Гарри Поттер", "Роулинг", 1997)
]

for book in books:
    print(book.title)
```

Закрепляется список объектов.

Пример 5. Поиск по полю

```
books = [
    Book("Хоббит", "Толкин", 1937),
    Book("Гарри Поттер", "Роулинг", 1997)
]

target = input("Введите автора: ")

for book in books:
    if book.author == target:
        print(book)
```

Закрепляется поиск по полю.

Пример 6. Фильтрация

```
books = [  
    Book("Книга 1", "Автор 1", 2000),  
    Book("Книга 2", "Автор 2", 2020),  
    Book("Книга 3", "Автор 3", 1990)  
]  
  
for book in books:  
    if book.year >= 2000:  
        print(book)
```

Закрепляется условие при работе с объектами.

Пример 7. Функция для объекта

```
def print_book(book):  
    print(book.title, "-", book.author)  
  
book = Book("Хоббит", "Толкин", 1937)  
  
print_book(book)
```

Закрепляется передача объекта в функцию.

7. Практические задания

Лёгкий уровень

1. Создать `dataclass` `Student` с полями `name`, `age`, `grade`.
2. Создать объект `Student` и вывести его.
3. Создать `dataclass` `Product` с полями `title`, `price`, `count`.
4. Создать объект `Product` и вывести отдельно его цену.
5. Изменить цену товара и вывести объект заново.

Средний уровень

1. Создать список из 3 товаров через `dataclass`.
2. Вывести названия всех товаров из списка.
3. Вывести только товары дешевле 1000.
4. Найти товар по названию.
5. Написать функцию, которая принимает товар и красиво выводит его данные.

Повышенный уровень

1. Создать `dataclass` `Movie` с полями `title`, `year`, `rating`. Вывести фильмы с рейтингом выше 8.
2. Создать список учеников и найти самого старшего.
3. Создать список товаров и посчитать общую стоимость всех товаров с учётом количества.

8. Контрольные вопросы

1. Что такое сущность?
2. Что такое поле?
3. Для чего нужен `dataclass` ?
4. Что нужно импортировать перед использованием `dataclass` ?
5. Зачем нужен символ `@` перед `dataclass` ?
6. Чем класс отличается от объекта?
7. Как создать объект `dataclass`?
8. Как получить значение поля?
9. Как изменить значение поля?
10. Почему объект удобнее списка без названий?
11. Можно ли хранить объекты в списке?
12. Как найти объект по одному из полей?

9. Краткий конспект в конце

Главная идея темы: `dataclass` помогает описывать сущности с понятными полями.

Основная конструкция:

```
from dataclasses import dataclass

@dataclass
class Phone:
    brand: str
    model: str
    price: int
```

Создание объекта:

```
phone = Phone("Samsung", "A55", 35000)
```

Обращение к полям:

```
print(phone.brand)
print(phone.price)
```

Список объектов:

```
phones = [
    Phone("Samsung", "A55", 35000),
    Phone("Apple", "iPhone 13", 60000)
]
```

Важно запомнить:

- `dataclass` нужно импортировать;
- перед `dataclass` ставится `@`;
- класс описывает поля;
- объект хранит конкретные данные;
- поля читаются через точку;
- список может хранить объекты;
- `dataclass` удобен для сущностей: товаров, книг, фильмов, учеников.

Частые ошибки:

- забыть импорт;
- забыть `@`;
- путать класс и объект;
- передать не все значения;
- перепутать порядок аргументов;
- обращаться к объекту как к списку;
- хранить связанные данные в разных списках.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- понимать, что такое сущность;
- описывать простые сущности через `dataclass`;
- импортировать `dataclass`;
- создавать объекты;
- читать поля объекта;
- изменять поля объекта;
- хранить объекты в списке;

- перебирать список объектов;
- искать объекты по полю;
- фильтровать объекты по условию;
- передавать объект в функцию.